

SQL

Сборник для быстрого повторения

Иванова Вероника

February 2026

Содержание

1	Основы SQL	4
1.1	Извлечение данных	4
1.2	ORDER BY — сортировка	4
1.3	WHERE — фильтрация данных	5
1.4	AND / OR / IN / NOT — логическая фильтрация	6
1.5	LIKE — шаблонный поиск	7
1.5.1	Экранирование (ESCAPE)	7
1.6	Регистрозависимость	8
1.6.1	В чем разница между CAST, CONVERT и BINARY	8
1.7	Вычисляемые поля	9
1.8	Объединение значений полей	10
2	Текстовые функции	11
2.1	Поиск подстроки и анализ длины строки	12
2.2	Извлечение части строки	13
2.3	Изменение регистра символов	14
2.4	Выравнивание и дополнение строки	15
2.5	Удаление части строки	15
2.5.1	Удаление пробелов	15
2.5.2	Функция TRIM и её расширенный синтаксис	16
2.6	Преобразование и модификация строки	17
3	Числовые функции и математические операции	18
3.1	Арифметические операции и преобразования чисел	18
3.2	Округление и форматирование чисел	19
3.3	Сравнение значений и случайные числа	20
3.3.1	Интервалы случайных чисел	21
3.4	Тригонометрические функции и работа с углами	22
4	Дата и время	23
4.1	Получение текущей даты и времени	24
4.2	Создание даты и времени	25
4.3	Извлечение компонентов даты и времени	26
4.4	Арифметика с датой и временем	28
4.5	Преобразование и вычисление	30

4.6	Форматирование даты и времени	31
4.6.1	Форматные спецификаторы и локализация	32

Основы SQL

1.1 Извлечение данных

Ключевое слово	Назначение	Пример
SELECT	Выбирает столбцы для вывода	SELECT name, age
FROM	Указывает таблицу-источник	FROM Users
LIMIT	Ограничивает число строк результата	LIMIT 5
LIMIT offset, n	Пропускает offset строк и выводит n строк	LIMIT 10, 5

Пример. Вывести названия и режиссёров первых 5 фильмов.

```
1 SELECT title, director
2 FROM Films
3 LIMIT 5;
```

1.2 ORDER BY — сортировка

Ключевое слово	Назначение	Пример
ORDER BY	Сортирует строки результирующей таблицы	ORDER BY release_date
ORDER BY ... ASC	Сортировка по возрастанию (по умолчанию)	ORDER BY name ASC
ORDER BY ... DESC	Сортировка по убыванию	ORDER BY place DESC

- можно сортировать по нескольким полям;
- можно использовать псевдонимы (AS) или исходные поля.
- В ORDER BY можно сортировать по псевдониму (AS) или по исходному полю
- Если поле переименовано в SELECT, можно сортировать по исходному, указав его как table.column (чтобы явно сослаться на столбец таблицы).

Пример. Отсортировать фильмы по названию в порядке убывания.

```
1 SELECT title AS name
2 FROM Films
3 ORDER BY name DESC;
```

1.3 WHERE — фильтрация данных

Назначение. Оператор `WHERE` отбирает строки, удовлетворяющие заданному условию.

- используется после `FROM` и до `ORDER BY` / `LIMIT`;
- условия задаются с помощью операторов сравнения;
- для проверки отсутствия значения используются `IS NULL` и `IS NOT NULL`;
- псевдонимы полей (`AS`) в `WHERE` использовать **нельзя**.

Ключевое слово	Назначение
<code>WHERE</code>	Фильтрация строк по условию
<code>WHERE ... =, !=, <>, <, <=, >, >=</code>	Сравнение с одиночным значением
<code>WHERE ... BETWEEN . AND .</code>	Проверка попадания в диапазон (включая границы)
<code>WHERE ... IS NULL</code>	Проверка на <code>NULL</code>
<code>WHERE ... IS NOT NULL</code>	Проверка на не- <code>NULL</code>

Пример. Выбрать песни с количеством прослушиваний больше 100000.

```
1 SELECT trackname, artist, streams
2 FROM Songs
3 WHERE streams > 100000;
```

1.4 AND / OR / IN / NOT — логическая фильтрация

Назначение. Логические операторы позволяют объединять и отрицать условия в WHERE.

Оператор	Назначение	Пример
WHERE cond1 AND cond2	Все условия истинны	WHERE streams > 50000 AND place <= 3
WHERE cond1 OR cond2	Хотя бы одно условие истинно	WHERE artist = 'Heart' OR artist = 'Kate Bush'
WHERE col IN (a, b, c)	Значение входит в список	WHERE artist IN ('Heart', 'Kate Bush')
WHERE NOT condition	Отрицание условия	WHERE NOT artist = 'The Sounds'
WHERE col NOT IN (a, b, c)	Значение не входит в список	WHERE artist NOT IN ('Heart', 'Kate Bush')

Приоритет операторов: IN, NOT IN → NOT → AND → OR

1.5 LIKE — шаблонный поиск

Назначение. Оператор LIKE используется для поиска строк по заданному шаблону.

- % — любая последовательность символов (0 и более);
- _ — ровно один любой символ;
- метасимволы могут располагаться в любом месте шаблона;
- оператор LIKE применяется только к строковым полям;
- по умолчанию LIKE не учитывает регистр.

1.5.1 Экранирование (ESCAPE)

Назначение. Используется для поиска самих метасимволов (% , _) как обычных символов.

- ключевое слово ESCAPE задаёт символ экранирования;
- символ после ESCAPE отключает специальное значение метасимвола.

Оператор	Назначение	Пример
WHERE col LIKE 'text%'	Строка начинается с заданного текста	WHERE artist LIKE 'The%'
WHERE col LIKE '%text%'	Строка содержит заданный текст	WHERE trackname LIKE '%You%'
WHERE col LIKE '___%'	Ровно заданное число символов в начале	WHERE trackname LIKE '___ %'
WHERE col LIKE pattern ESCAPE 'x'	Поиск самого метасимвола	WHERE value LIKE '__/%' ESCAPE '/'

1.6 Регистрозависимость

Назначение. В SQL сравнение строк по умолчанию регистронезависимо (это относится как к LIKE, так и к оператору =).

- если регистр важен, строку или шаблон необходимо явно привести к бинарному виду;
- после приведения сравнение выполняется с учетом регистра.

Ключевое слово	Назначение	Пример
CAST(... AS BINARY)	Приведение строки к бинарному виду для учета регистра	LIKE CAST('%You%' AS BINARY)
CONVERT(..., BINARY)	Альтернативная форма приведения	LIKE CONVERT('%You%', BINARY)
BINARY	Сокращённая форма, чаще используется в рабочем коде	LIKE BINARY '%You%'

1.6.1 В чем разница между CAST, CONVERT и BINARY

- CAST(... AS BINARY) — стандартный SQL-синтаксис, универсальный и наглядный;
- CONVERT(..., BINARY) — альтернативная форма, часто используемая в MySQL;
- в задачах регистрозависимости CAST и CONVERT эквивалентны и выполняют одно и то же действие;
- использование BINARY напрямую — сокращённая форма:
 - чаще встречается в рабочем (боевом) коде;
 - менее универсальна;
 - по смыслу эквивалентна CAST и CONVERT.

Идея. SQL по умолчанию не учитывает регистр, поэтому, если регистр важен, это необходимо указать явно — через BINARY.

1.7 Вычисляемые поля

Назначение. Вычисляемые поля — это поля, значения которых не хранятся в таблице, а вычисляются в запросе на основе других полей.

- вычисляются при выполнении `SELECT`;
- могут включать выражения и функции;
- в таблице базы данных не сохраняются;
- поддерживаются операторы `+`, `-`, `*`, `/`;
- если один из операндов равен `NULL`, результат равен `NULL`;
- псевдонимы вычисляемых полей **нельзя** использовать в `WHERE`
- псевдонимы вычисляемых полей **можно** использовать в `ORDER BY`

Пример. Магазин мерча «VIBE STORE». Для каждого товара известны цена и количество продаж. Рассчитаем выручку по каждому товару.

Запрос:

```
1 SELECT item,  
2     price * quantity AS revenue  
3 FROM VibeStore;
```

Исходные данные:

item	price	quantity
Vibe Hoodie	50	120
Soft Vibe Socks	10	340

Результат:

item	revenue
Vibe Hoodie	6000
Soft Vibe Socks	3400

1.8 Объединение значений полей

Назначение. Используется для объединения значений полей в одно строковое поле.

Ключевое слово	Назначение	Пример
CONCAT(col1, col2, ...)	Объединяет значения полей в одну строку без разделителя по умолчанию (например: Иванова + Вероника → ИвановаВероника)	CONCAT(artist, trackname)
CONCAT(col1, ' ', col2)	Объединяет значения с явно заданным разделителем, который указывается вручную	CONCAT(artist, ' - ', trackname)
CONCAT_WS(sep, col1, col2, ...)	Объединяет значения и вставляет разделитель sep между ними (например: Иванова, Вероника → Иванова Вероника)	CONCAT_WS(' ', artist, trackname)

- все аргументы неявно приводятся к строке;
- если хотя бы один аргумент NULL, CONCAT возвращает NULL;
- CONCAT_WS игнорирует NULL-значения (кроме разделителя).

Пример. Пусть в таблице хранится информация о товарах: тип одежды, цвет и размер. Необходимо получить удобное для вывода описание товара в одной строке.

Запрос:

```
1 SELECT CONCAT_WS(' ', item, color,  
2         CONCAT('size ', size)) AS product  
3 FROM Clothes;
```

Исходные данные:

item	color	size
Hoodie	black	M
T-shirt	white	L

Результат:

product
Hoodie, black, size M
T-shirt, white, size L

Текстовые функции

Чтобы упростить навигацию и сделать материал более удобным для повторения, текстовые функции в данном разделе сгруппированы по типу выполняемых операций.

В рамках раздела рассматриваются следующие группы текстовых функций:

Извлечение части строки — функции, позволяющие получить фрагмент строки по позиции, длине или разделителю: `LEFT()` , `RIGHT()` , `SUBSTRING()` , `SUBSTRING_INDEX()` .

Поиск и анализ строки — функции, используемые для определения позиции подстроки и длины строки: `LOCATE()` , `CHAR_LENGTH()` .

Изменение регистра символов — функции для приведения текста к одному регистру: `LOWER()` , `UPPER()` .

Удаление пробелов — функции для очистки строк от ведущих и замыкающих пробелов: `TRIM()` , `LTRIM()` , `RTRIM()` .

Преобразование и модификация строки — функции, изменяющие структуру или порядок символов: `REPLACE()` , `REVERSE()` , `REPEAT()` .

Выравнивание и дополнение строки — функции для искусственного изменения длины строки: `LPAD()` , `RPAD()` .

2.1 Поиск подстроки и анализ длины строки

Назначение. Определить позицию подстроки внутри строки и вычислить длину строкового значения.

Функции: `LOCATE()` `CHAR_LENGTH()` .

Функция	Назначение
<code>LOCATE(substr, str, start)</code>	Возвращает позицию первого вхождения подстроки substr в строке str (нумерация с 1). start задаёт позицию начала поиска (необязательный). Если подстрока не найдена, возвращается 0. Поиск выполняется без учёта регистра.
<code>CHAR_LENGTH(str)</code>	Возвращает количество символов в строке str . При передаче <code>NULL</code> возвращается <code>NULL</code> .

Пример. Определим длину строки и позицию подстроки в текстовых значениях.

```
1 SELECT
2   text,
3   CHAR_LENGTH(text)      AS text_length,
4   LOCATE('vibe', text)   AS vibe_pos,
5   LOCATE('swag', text, 3) AS swag_pos
6 FROM Data;
```

Результат:

text	text_length	vibe_pos	swag_pos
vibe_mode	9	1	0
mega_swag	9	0	6
ultra_vibe	10	7	0
vibe-swag-style	15	1	6

2.2 Извлечение части строки

Назначение. Быстро получить фрагмент строки по позиции, длине или разделителю.

Функции: `LEFT()` `RIGHT()` `SUBSTRING()` `SUBSTRING_INDEX()` .

Функция	Назначение
<code>LEFT(str, n)</code>	Возвращает первые <code>n</code> символов строки <code>str</code> . Если <code>n ></code> длины строки — возвращается вся строка. Если <code>n ≤ 0</code> — возвращается пустая строка.
<code>RIGHT(str, n)</code>	Аналогичное поведение, но возвращает последние символы.
<code>SUBSTRING(str, start, len)</code>	Возвращает подстроку из <code>str</code> , начиная с позиции <code>start</code> (нумерация с 1), длиной <code>len</code> . Допускается отрицательное значение <code>start</code> — отсчёт с конца строки.
<code>SUBSTRING_INDEX(str, delim, k)</code>	Возвращает часть строки <code>str</code> до <code>k</code> -го появления разделителя <code>delim</code> . Если <code>k < 0</code> , извлечение происходит с конца строки. Если разделитель отсутствует, возвращается вся строка.

Пример. Извлечём разные части строки из текстовых значений с помощью функций

```
1 SELECT
2   text,
3   LEFT(text, 4)           AS left_part,
4   RIGHT(text, 4)          AS right_part,
5   SUBSTRING(text, 7, 4)   AS mid_part,
6   SUBSTRING_INDEX(text, '-', 1) AS first_part,
7   SUBSTRING_INDEX(text, '-', -1) AS last_part
8 FROM Data;
```

Результат:

text	left_part	right_part	mid_part	first_part	last_part
vibe_mode	vibe	mode	ode	vibe_mode	vibe_mode
mega_swag	mega	swag	wag	mega_swag	mega_swag
ultra_vibe	ultr	vibe	vibe	ultra_vibe	ultra_vibe
vibe-swag-style	vibe	tyle	wag-	vibe	style

2.3 Изменение регистра символов

Назначение. Приведение текста к одному регистру — нижнему или верхнему.

Функции: `LOWER()` `UPPER()` .

Функция	Назначение
<code>LOWER(str)</code>	Преобразует все буквенные символы строки str в нижний регистр.
<code>UPPER(str)</code>	Преобразует все буквенные символы строки str в верхний регистр.

При NULL возвращают NULL

Пример.

```
1 SELECT
2   text,
3   LOWER(text) AS lower_text,
4   UPPER(text) AS upper_text
5 FROM Data;
```

Результат:

text	lower_text	upper_text
Mega_Swag	mega_swag	MEGA_SWAG
ULTRA_vibe	ultra_vibe	ULTRA_VIBE

2.4 Выравнивание и дополнение строки

Назначение. Искусственное изменение длины строки за счёт добавления символов слева или справа до заданной длины.

Функции: `LPAD()` `RPAD()` .

Функция	Назначение
<code>LPAD(str, len, pad)</code>	Дополняет строку <code>str</code> слева символами из строки <code>pad</code> до длины <code>len</code> слева. Если итоговая длина превышает <code>len</code> , строка обрезается справа.
<code>RPAD(str, len, pad)</code>	Аналогично, но все справа.

Пример.

```
1 SELECT
2   text,
3   LPAD(text, 12, ' _') AS left_padded,
4   RPAD(text, 12, ' _') AS right_padded
5 FROM Data;
```

Результат:

text	left_padded	right_padded
vibe	_____vibe	vibe_____

2.5 Удаление части строки

2.5.1 Удаление пробелов

Назначение. Очистка строк от лишних символов в начале и/или конце строки.

Функции: `LTRIM()` `RTRIM()` `TRIM()` .

Функция	Назначение
<code>LTRIM(str)</code>	Удаляет пробелы из начала строки <code>str</code> .
<code>RTRIM(str)</code>	Удаляет пробелы из конца строки <code>str</code> .
<code>TRIM(str)</code>	Удаляет пробелы из начала и конца строки <code>str</code> (поведение по умолчанию).

2.5.2 Функция TRIM и её расширенный синтаксис

Назначение. Удаляет указанную **подстроку** из начала и/или конца строки.

Синтаксис:

TRIM(LEADING | TRAILING | BOTH <подстрока> FROM <строка>)

Вариант	Что делает
TRIM(LEADING ...)	Удаляет подстроку только из начала строки.
TRIM(TRAILING ...)	Удаляет подстроку только из конца строки.
TRIM(BOTH ...)	Удаляет подстроку из начала и конца строки.
TRIM(str)	Удаляет пробелы из начала и конца строки (если подстрока не указана).

Важно. Функция TRIM() удаляет подстроку *только с краёв строки* и *только пока она встречается подряд*. Внутренняя часть строки при этом не затрагивается.

Пример. Удалим подстроку из начала и/или конца строки. Удаление происходит *только по краям строки* и *только пока подстрока встречается подряд*.

```
1 SELECT
2     text,
3     TRIM(LEADING '-' FROM text) AS trim_leading,
4     TRIM(TRAILING '-' FROM text) AS trim_trailing,
5     TRIM(BOTH      '-' FROM text) AS trim_both,
6     TRIM('-' FROM text)           AS trim_default,
7     TRIM('+-' FROM text)          AS trim_mixed
8 FROM (
9     SELECT '--vibe--' AS text UNION ALL
10    SELECT '+-+vibe+-+' AS text
11 ) AS Data;
```

Результат:

text	trim_leading	trim_trailing	trim_both	trim_default	trim_mixed
-vibe-	vibe-	-vibe	vibe	vibe	-vibe-
+--+vibe+-+	+--+vibe+-+	+--+vibe+-+	+--+vibe+-+	+--+vibe+-+	+vibe+-+

2.6 Преобразование и модификация строки

Назначение. Изменение структуры или порядка символов в строке.

Функции: REPLACE() REVERSE() REPEAT() .

Функция	Назначение
REPLACE(str, from, to)	В строке str заменяет все вхождения подстроки from на подстроку to . Если подстрока from в строке отсутствует, функция возвращает исходную строку без изменений.
REVERSE(str)	Возвращает строку str с обратным порядком символов.
REPEAT(str, n)	Возвращает строку, состоящую из n повторений строки str . Если $n \leq 0$, возвращается пустая строка.

Пример.

```
1 SELECT
2   text,
3   REPLACE(text, 'vibe', 'swag') AS replaced,
4   REVERSE(text)                  AS reversed,
5   REPEAT(text, 2)                AS repeated
6 FROM Data;
```

Результат:

text	replaced	reversed	repeated
vibe_mode	swag_mode	edom_ebiv	vibe_modevibe_mode
mega_vibe	mega_swag	ebiv_agem	mega_vibemega_vibe

Числовые функции и математические операции

3.1 Арифметические операции и преобразования чисел

Назначение. Выполнение целочисленного деления, вычисление остатка, нахождение модуля числа, возведение в степень и вычисление квадратного корня.

Функции: `DIV` `MOD` `%` `ABS()` `POW()` `SQRT()`.

Функция	Назначение
a DIV b	Целочисленное деление числа a на число b (возвращается только целая часть).
a MOD b	Возвращает остаток от деления числа a на число b.
a % b	Альтернативная форма вычисления остатка от деления.
ABS(num)	Возвращает модуль числа num.
POW(num, degree)	Возводит число num в степень degree.
SQRT(num)	Возвращает квадратный корень числа num. Если num < 0, возвращается NULL.

Пример.

```
1 SELECT
2     11 DIV 2 AS div_result,
3     11 MOD 2 AS mod_result,
4     11 % 2 AS percent_result,
5     ABS(-5) AS abs_result,
6     POW(2, 3) AS pow_result,
7     SQRT(9) AS sqrt_result;
```

Действие:

11 DIV 2	11 MOD 2	11 % 2	ABS(-5)	POW(2,3)	SQRT(9)
----------	----------	--------	---------	----------	---------

Результат:

div_result	mod_result	percent_result	abs_result	pow_result	sqrt_result
5	1	1	5	8	3

3.2 Округление и форматирование чисел

Назначение. Округление чисел по различным правилам и форматирование числовых значений для вывода.

Функции: `ROUND()` `FLOOR()` `CEILING()` `FORMAT()` .

Функция	Назначение
<code>ROUND(num, d)</code>	Округляет число <code>num</code> до <code>d</code> знаков после запятой по математическим правилам. Если <code>d</code> не указан, выполняется округление до целого числа.
<code>FLOOR(num)</code>	Округляет число <code>num</code> вниз (в меньшую сторону).
<code>CEILING(num)</code>	Округляет число <code>num</code> вверх (в большую сторону).
<code>FORMAT(num, d)</code>	Округляет число <code>num</code> до <code>d</code> знаков и форматирует его с разделением разрядов. Результат возвращается в виде строки.

Пример.

```
1 SELECT
2     ROUND(23.165, 2)          AS round_pos,
3     ROUND(-23.165, 2)       AS round_neg,
4     FLOOR(-11.22)           AS floor_result,
5     CEILING(-11.22)         AS ceiling_result,
6     FORMAT(1000000.1653, 2) AS format_result,
7     FORMAT(ROUND(1234, -1), 0) AS format_tens;
```

Результат:

round_pos	round_neg	floor_result	ceiling_result	format_result	format_tens
23.17	-23.17	-12	-11	1,000,000.17	1,230

Важно.

- `ROUND()` возвращает числовое значение (используется для вычислений).
- `FORMAT()` возвращает строку (используется для отображения).

3.3 Сравнение значений и случайные числа

Назначение. Определение минимального и максимального значения среди нескольких аргументов, а также генерация случайных чисел.

Функции: `LEAST()` `GREATEST()` `RAND()` .

Функция	Назначение
<code>LEAST(a, b, ...)</code>	Возвращает наименьшее значение среди переданных аргументов.
<code>GREATEST(a, b, ...)</code>	Возвращает наибольшее значение среди переданных аргументов.
<code>RAND()</code>	Генерирует вещественное число в интервале $[0; 1)$.
<code>RAND(seed)</code>	Генерирует детерминированное случайное число при фиксированном seed. При одинаковых seed одинаковое число

Пример.

```
1 SELECT
2     LEAST(2, 5, 1, 9)      AS least_result,
3     GREATEST(2, 5, 1, 9)  AS greatest_result,
4     RAND(2)               AS rand_seed;
```

Результат:

least_result	greatest_result	rand_seed
1	9	0.6555866465490187

3.3.1 Интервалы случайных чисел

Назначение. По умолчанию функция `RAND()` возвращает вещественное число в интервале $[0; 1)$. Однако на практике часто требуется получить случайное число в заданных границах. Пусть требуется получить число в интервале $[a; b)$. Тогда используется выражение:

$$a + RAND() \cdot (b - a)$$

Идея. `RAND()` генерирует число от 0 до 1. Умножение на $(b - a)$ масштабирует интервал до нужной длины, а прибавление a сдвигает его к нижней границе.

Вывод.

$$[a; b) \Rightarrow a + RAND() \cdot (b - a)$$

Если требуется получить число в интервале $[a; b]$ (включена верхняя граница), используется:

$$a + RAND() \cdot (b - a + 1)$$

$$[a; b] \Rightarrow a + RAND() \cdot (b - a + 1)$$

Пример 1. Получение числа в интервале $[10; 20)$.

```
1 SELECT 10 + RAND() * (20 - 10) AS random_value;
```

Результат (пример):

random_value
14.3729182746

Пример 2. Получение целого числа в интервале $[1; 10]$.

```
1 SELECT FLOOR(1 + RAND() * (10 - 1 + 1)) AS random_int;
```

Результат (пример):

random_int
7

3.4 Тригонометрические функции и работа с углами

Назначение. Преобразование значений между градусами и радианами, а также вычисление тригонометрических функций.

Функции: `DEGREES()` `RADIANS()` `PI()` `SIN()` `COS()` `TAN()` .

Функция	Назначение
DEGREES(x)	Преобразует значение x из радиан в градусы.
RADIANS(x)	Преобразует значение x из градусов в радианы.
PI()	Возвращает значение числа π .
SIN(x)	Возвращает синус угла x (в радианах).
COS(x)	Возвращает косинус угла x (в радианах).
TAN(x)	Возвращает тангенс угла x (в радианах).

Пример.

```
1 SELECT
2     PI()                AS pi_value,
3     ROUND(RADIANS(180), 6) AS rad_value,
4     DEGREES(PI())       AS deg_value,
5     SIN(PI())           AS sin_value,
6     COS(PI())           AS cos_value,
7     TAN(PI())           AS tan_value;
```

Действие:

π	$180^\circ \rightarrow \text{рад}$	$\pi \rightarrow \text{град}$	$\sin(\pi)$	$\cos(\pi)$	$\tan(\pi)$
-------	------------------------------------	-------------------------------	-------------	-------------	-------------

Результат:

pi_value	rad_value	deg_value	sin_value	cos_value	tan_value
3.141593	3.141593	180	0	-1	0

Важно.

- Тригонометрические функции принимают аргументы в радианах.
- Функция `PI()` возвращает значение числа π с точностью до шести знаков после запятой.

Дата и время

Чтобы упростить навигацию и сделать материал более удобным, функции работы с датой и временем в данном разделе сгруппированы по типу выполняемых операций.

В рамках раздела рассматриваются следующие группы функций:

Получение текущей даты и времени — функции для определения текущей даты, времени и даты-времени в локальном часовом поясе и в UTC:

`NOW()`, `CURDATE()`, `CURTIME()`, `UTC_TIMESTAMP()`, `UTC_DATE()`, `UTC_TIME()`.

Создание даты и времени — функции для формирования даты, времени и объединения их в единое значение:

`MAKEDATE()`, `MAKETIME()`, `TIMESTAMP()`.

Извлечение компонентов даты и времени — функции, позволяющие получить отдельные части даты или времени:

`DATE()`, `TIME()`, `hour()`, `minute()`, `second()`, `microsecond()`, `year()`, `month()`, `day()`, `dayofyear()`, `week()`, `weekday()`, `quarter()`, `dayname()`, `monthname()`.

Арифметика с датой и временем — операции прибавления, вычитания и вычисления разницы между датами и временем:

`interval`, `adddate()`, `subdate()`, `addtime()`, `subtime()`, `datediff()`, `timediff()`, `timestampdiff()`.

Преобразование и вычисление — функции для перевода временных значений в числовые формы и выполнения вспомогательных вычислений:

`time_to_sec()`, `sec_to_time()`, `to_days()`, `to_seconds()`, `last_day()`.

Форматирование даты и времени — функции для изменения формата отображения даты и времени и обратного преобразования строк:

`date_format()`, `time_format()`, `str_to_date()`, `get_format()`.

4.1 Получение текущей даты и времени

Назначение. Определение текущей даты, времени и даты-времени в локальном часовом поясе сервера или в UTC.

UTC (Coordinated Universal Time) — всемирное координированное время. Это нулевой часовой пояс, относительно которого рассчитываются остальные. Использование UTC позволяет избежать проблем с часовыми поясами и переходом на летнее/зимнее время.

Функции:

`NOW()`, `CURDATE()`, `CURTIME()`, `UTC_TIMESTAMP()`, `UTC_DATE()`, `UTC_TIME()`.

Функция	Назначение
<code>NOW()</code>	Возвращает текущие дату и время в локальном часовом поясе сервера.
<code>CURDATE()</code>	Возвращает текущую дату в локальном часовом поясе сервера.
<code>CURTIME()</code>	Возвращает текущее время в локальном часовом поясе сервера.
<code>UTC_TIMESTAMP()</code>	Возвращает текущие дату и время в UTC (нулевой часовой пояс).
<code>UTC_DATE()</code>	Возвращает текущую дату в UTC.
<code>UTC_TIME()</code>	Возвращает текущее время в UTC.

Пример. Получить текущие дату и время в локальном часовом поясе и в UTC.

```
1 SELECT NOW(),
2        CURDATE(),
3        CURTIME(),
4        UTC_TIMESTAMP(),
5        UTC_DATE(),
6        UTC_TIME();
```

Результат:

<code>NOW()</code>	<code>CURDATE()</code>	<code>CURTIME()</code>	<code>UTC_TIMESTAMP()</code>	<code>UTC_DATE()</code>	<code>UTC_TIME()</code>
2026-02-12 23:41:15	2026-02-12	23:41:15	2026-02-12 20:41:15	2026-02-12	20:41:15

4.2 Создание даты и времени

Назначение. Формирование даты, времени и объединение их в единое значение типа DATETIME.

Создание даты и времени — функции для формирования даты, времени и объединения их в единое значение:

MAKEDATE() , MAKETIME() , TIMESTAMP() .

Функция	Назначение	Пример
MAKEDATE(year, day)	Создаёт дату по номеру дня в году.	MAKEDATE(2023, 255)
MAKETIME(hour, min, sec)	Создаёт временное значение из часов, минут и секунд.	MAKETIME(12, 30, 15)
TIMESTAMP(date, time)	Объединяет дату и время в одно значение DATETIME.	TIMESTAMP('2023-10-20', '08:00:00')

Пример. Создать дату, время и объединить их в единое значение.

```
1 SELECT MAKEDATE(2023, 255),  
2     MAKETIME(12, 30, 15),  
3     TIMESTAMP('2023-10-20', '08:00:00');
```

Результат:

MAKEDATE(2023, 255)	MAKETIME(12, 30, 15)	TIMESTAMP('2023-10-20','08:00:00')
2023-09-12	12:30:15	2023-10-20 08:00:00

4.3 Извлечение компонентов даты и времени

Назначение. Получение отдельных частей даты или времени: года, месяца, дня, часов, минут, секунд, микросекунд, номера недели, дня недели, квартала и текстовых названий.

Функции:

DATE(), TIME(), HOUR(), MINUTE(), SECOND(), MICROSECOND(), YEAR(), MONTH(), DAY(), DAYOFYEAR(), WEEK(), WEEKDAY(), QUARTER(), DAYNAME(), MONTHNAME().

Функция	Назначение	Пример
DATE(expr)	Извлекает дату из значения DATETIME.	DATE('2023-10-20 12:30:15')
TIME(expr)	Извлекает время из значения DATETIME.	TIME('2023-10-20 12:30:15')
HOUR(time)	Возвращает часы.	HOUR('12:30:15')
MINUTE(time)	Возвращает минуты.	MINUTE('12:30:15')
SECOND(time)	Возвращает секунды.	SECOND('12:30:15')
MICROSECOND(time)	Возвращает микросекунды.	MICROSECOND('12:30:15.123456')
YEAR(date)	Возвращает год.	YEAR('2023-10-20')
MONTH(date)	Возвращает номер месяца.	MONTH('2023-10-20')
DAY(date)	Возвращает день месяца.	DAY('2023-10-20')
DAYOFYEAR(date)	Номер дня в году (1–366).	DAYOFYEAR('2023-10-20')
WEEK(date)	Номер недели в году.	WEEK('2023-10-20')
WEEKDAY(date)	Номер дня недели (0 = понедельник).	WEEKDAY('2023-10-20')
QUARTER(date)	Номер квартала (1–4).	QUARTER('2023-10-20')
DAYNAME(date)	Название дня недели.	DAYNAME('2023-10-20')
MONTHNAME(date)	Название месяца.	MONTHNAME('2023-10-20')

Пример 1. Извлечь основные числовые компоненты.

```
1 SELECT DATE('2023-10-20 12:30:15'),
2        TIME('2023-10-20 12:30:15'),
3        MONTH('2023-10-20'),
4        DAY('2023-10-20'),
5        HOUR('12:30:15'),
6        SECOND('12:30:15'),
7        MICROSECOND('12:30:15.123456');
```

Результат 1:

DATE()	TIME()	MONTH()	DAY()	HOUR()	SECOND()	MICROSECOND()
2023-10-20	12:30:15	10	20	12	15	123456

Пример 2. Извлечь календарные характеристики даты.

```
1 SELECT DAYOFYEAR('2023-10-20'),
2        WEEK('2023-10-20'),
3        WEEKDAY('2023-10-20'),
4        QUARTER('2023-10-20'),
5        DAYNAME('2023-10-20'),
6        MONTHNAME('2023-10-20');
```

Результат 2:

DAYOFYEAR()	WEEK()	WEEKDAY()	QUARTER()	DAYNAME()	MONTHNAME()
293	42	4	4	Friday	October

4.4 Арифметика с датой и временем

Назначение. Прибавление, вычитание и вычисление разницы между датами и временными значениями.

Арифметика с датой и временем — операции прибавления, вычитания и вычисления разницы между датами и временем:

`INTERVAL` , `ADDDATE()` , `SUBDATE()` , `ADDTIME()` , `SUBTIME()` ,
`DATEDIFF()` , `TIMEDIFF()` , `TIMESTAMPDIFF()` .

Функция	Назначение	Пример
<code>INTERVAL</code>	Задаёт временной интервал.	<code>INTERVAL 5 DAY</code>
<code>ADDDATE(date, interval)</code>	Прибавляет интервал к дате.	<code>ADDDATE('2023-10-20', INTERVAL 5 DAY)</code>
<code>SUBDATE(date, interval)</code>	Вычитает интервал из даты.	<code>SUBDATE('2023-10-20', INTERVAL 5 DAY)</code>
<code>ADDTIME(datetime, time)</code>	Прибавляет время к DATETIME.	<code>ADDTIME('2023-10-20 10:00:00', '02:00:00')</code>
<code>SUBTIME(datetime, time)</code>	Вычитает время из DATETIME.	<code>SUBTIME('2023-10-20 10:00:00', '02:00:00')</code>
<code>DATEDIFF(date1, date2)</code>	Разница между датами в днях.	<code>DATEDIFF('2023-10-25', '2023-10-20')</code>
<code>TIMEDIFF(dt1, dt2)</code>	Разница между датой-временем.	<code>TIMEDIFF('2023-10-20 12:00:00', '2023-10-20 10:00:00')</code>
<code>TIMESTAMPDIFF(unit, dt1, dt2)</code>	Разница в указанных единицах измерения.	<code>TIMESTAMPDIFF(HOUR, '2023-10-20 10:00:00', '2023-10-20 15:00:00')</code>

Пример. Выполнить арифметические операции с датой и временем.

```
1 SELECT ADDDATE('2023-10-20', INTERVAL 5 DAY),  
2         SUBDATE('2023-10-20', INTERVAL 5 DAY),  
3         ADDTIME('2023-10-20 10:00:00', '02:00:00'),  
4         DATEDIFF('2023-10-25', '2023-10-20'),  
5         TIMESTAMPDIFF(HOUR,  
6                        '2023-10-20 10:00:00',  
7                        '2023-10-20 15:00:00');
```

Результат:

ADDDATE()	SUBDATE()	ADDTIME()	DATEDIFF()	TIMESTAMPDIFF()
2023-10-25	2023-10-15	2023-10-20 12:00:00	5	5

4.5 Преобразование и вычисление

Назначение. Преобразование даты и времени в числовые значения и выполнение вспомогательных вычислений.

Преобразование и вычисление — функции для перевода временных значений в числовую форму и получения вспомогательных характеристик:

`TIME_TO_SEC()` , `SEC_TO_TIME()` , `TO_DAYS()` , `TO_SECONDS()` , `LAST_DAY()` .

Функция	Назначение	Пример
<code>TIME_TO_SEC(time)</code>	Преобразует время в количество секунд.	<code>TIME_TO_SEC('01:30:00')</code>
<code>SEC_TO_TIME(sec)</code>	Преобразует секунды во временной формат.	<code>SEC_TO_TIME(5400)</code>
<code>TO_DAYS(date)</code>	Возвращает количество дней от нулевой даты.	<code>TO_DAYS('2023-10-20')</code>
<code>TO_SECONDS(date)</code>	Возвращает количество секунд от нулевой даты.	<code>TO_SECONDS('2023-10-20')</code>
<code>LAST_DAY(date)</code>	Возвращает последний день месяца.	<code>LAST_DAY('2023-10-20')</code>

Пример. Выполнить преобразование времени и получить числовые характеристики даты.

```
1 SELECT TIME_TO_SEC('01:30:00'),  
2        SEC_TO_TIME(5400),  
3        TO_DAYS('2023-10-20'),  
4        TO_SECONDS('2023-10-20'),  
5        LAST_DAY('2023-10-20');
```

Результат:

<code>TIME_TO_SEC()</code>	<code>SEC_TO_TIME()</code>	<code>TO_DAYS()</code>	<code>TO_SECONDS()</code>	<code>LAST_DAY()</code>
5400	01:30:00	739173	63833683200	2023-10-31

4.6 Форматирование даты и времени

Назначение. Изменение формата отображения даты и времени и преобразование строк в значения типа DATE/DATETIME.

Форматирование даты и времени — функции для изменения формата отображения даты и времени и обратного преобразования строк:

`DATE_FORMAT()` , `TIME_FORMAT()` , `STR_TO_DATE()` , `GET_FORMAT()` .

Функция	Назначение	Пример
<code>DATE_FORMAT(date, format)</code>	Форматирует дату по заданному шаблону.	<code>DATE_FORMAT('2023-10-20', '%d.%m.%Y')</code>
<code>TIME_FORMAT(time, format)</code>	Форматирует время по заданному шаблону.	<code>TIME_FORMAT('12:30:15', '%H:%i')</code>
<code>STR_TO_DATE(str, format)</code>	Преобразует строку в дату по шаблону.	<code>STR_TO_DATE('20.10.2023', '%d.%m.%Y')</code>
<code>GET_FORMAT(type, locale)</code>	Возвращает стандартный шаблон формата.	<code>GET_FORMAT(DATE, 'EUR')</code>

Примечание 1 (особенности аргументов функций).

`DATE_FORMAT()` принимает значение типа DATE или DATETIME. Если передать только дату, время считается равным 00:00:00.

`TIME_FORMAT()` предназначена только для работы со временем. При передаче даты возможны ошибки или некорректный результат.

`STR_TO_DATE()` требует полного соответствия строки и шаблона. Если хотя бы одна часть строки не совпадает с форматом, функция вернёт NULL.

В спецификаторе %H часы ожидаются в диапазоне 00–23. Если указано число без ведущего нуля, необходимо использовать соответствующий формат (например, %H корректно обработает 05, но при сложных шаблонах возможны несоответствия).

Пример. Отформатировать дату и выполнить обратное преобразование.

```
1 SELECT DATE_FORMAT('2023-10-20', '%d.%m.%Y'),
2        TIME_FORMAT('12:30:15', '%H:%i'),
3        STR_TO_DATE('20.10.2023', '%d.%m.%Y'),
4        GET_FORMAT(DATE, 'EUR');
```

Результат:

DATE_FORMAT()	TIME_FORMAT()	STR_TO_DATE()	GET_FORMAT()
20.10.2023	12:30	2023-10-20	%Y-%m-%d

4.6.1 Форматные спецификаторы и локализация

Назначение. В функциях `DATE_FORMAT()` и `STR_TO_DATE()` используются специальные форматные коды (спецификаторы), которые начинаются с символа `%`. Каждый код соответствует определённому компоненту даты или времени.

Код	Назначение
%Y	Год (4 цифры)
%y	Год (2 цифры)
%m	Номер месяца (01–12)
%c	Номер месяца без ведущего нуля
%M	Полное название месяца
%b	Сокращённое название месяца
%d	День месяца (01–31)
%e	День месяца без ведущего нуля
%H	Часы (00–23)
%h	Часы (01–12)
%i	Минуты (00–59)
%s	Секунды (00–59)
%W	Полное название дня недели
%a	Сокращённое название дня недели

Пример. Отформатировать дату и извлечь отдельные компоненты.

```
1 SELECT DATE_FORMAT('2023-10-20 17:05:00',
2             'Сегодня %W, %d %M %Y года, %H:%i'),
3         STR_TO_DATE('Сегодня 20.10.2023 в 5 часов',
4             'Сегодня %d.%m.%Y в %H часов');
```

Результат:

DATE_FORMAT()	STR_TO_DATE()
Сегодня Friday, 20 October 2023 года, 17:05	2023-10-20 05:00:00

Примечание 1. При использовании функции `STR_TO_DATE()` строка и шаблон должны полностью совпадать. Если текст шаблона отличается от исходной строки, функция вернёт значение NULL.

Примечание 2 (локализация). Названия месяцев и дней недели зависят от системной переменной `lc_time_names`. По умолчанию часто используется английская локаль (`en_US`). Чтобы изменить язык отображения на русский, необходимо выполнить команду:

```
1 SET lc_time_names = 'ru_RU';
```

Команда должна быть выполнена **перед** использованием функций `DATE_FORMAT()`, `DAYNAME()` или `MONTHNAME()`.

Пример.

```
1 SET lc_time_names = 'ru_RU';
2
3 SELECT DATE_FORMAT('2023-10-20', '%W, %d %M %Y'),
4         DAYNAME('2023-10-20'),
5         MONTHNAME('2023-10-20');
```

Результат:

DATE_FORMAT()	DAYNAME()	MONTHNAME()
Пятница, 20 Октября 2023	Пятница	Октября

Важно.

Команда `SET lc_time_names` действует только в текущей сессии. После перезапуска соединения язык может вернуться к значению по умолчанию. Если локаль указана неверно, будет выдана ошибка.